PROJECT PLAN XISD5319

TEAM NAME - KNIGHTS

MEMBERS

ST10447100 – UNATHI MGANDELA

S10438928 – CLARITY MASUKU

ST10462532 – SIBUSISO MABENA

ST10450008-PHUMUDZO MUNYAI

Nanny-App

Care • Connect • Comfort

Contents

1. PROBLEM DOMAIN

Many South African parents struggle to locate safe, dependable, and convenient daycare, especially at short notice. Traditional approaches (referrals, social media posts, neighbourhood suggestions, or informal agencies) frequently lack sufficient verification of nannies, secure communication channels, background checks, and secure payment mechanisms. This puts children at risk, causes stress, inconvenience, and raises concerns about their safety.

Key problems identified:

- There is a lack of trustworthy verification mechanisms for nannies.
- The search and booking process is inefficient and time-consuming.
- There's no safe in-app communication or real-time availability tracking.
- Unsafe or imprecise payment processing.
- Difficulty managing reservations, cancellations, and reviews.
- Qualified nannies have limited prospects for career advancement.

Nanny-App solves these concerns by developing a digital platform that links parents with verified nannies in a secure, convenient, and trustworthy environment.

2. SOLUTION DOMAIN

2.1 Active Actors

- Parent (Client): Signs up, looks for nannies, reads profiles, makes reservations, communicates, pays, and writes reviews.
- Nanny (Service Provider): Establishes and maintains a professional profile, determines availability, answers booking inquiries, interacts with parents, and gets paid.
- Administrator (Upcoming improvement): Oversees platform compliance, dispute resolution, and user verification.

2.2 Passive Actors

- Payment Gateway (Paystack)
- Notification Service (Email / SMS / Push Notifications)
- Database System

2.3 Main functions

Parent Functions:

- Managing profiles and registering users.
- Nannies can be found and filtered by location, availability, rating, and rate.
- View thorough evaluations and profiles of nannies.
- Make and oversee reservations.
- Safe in-app communication.
- Safe online transactions.
- View the booking history.

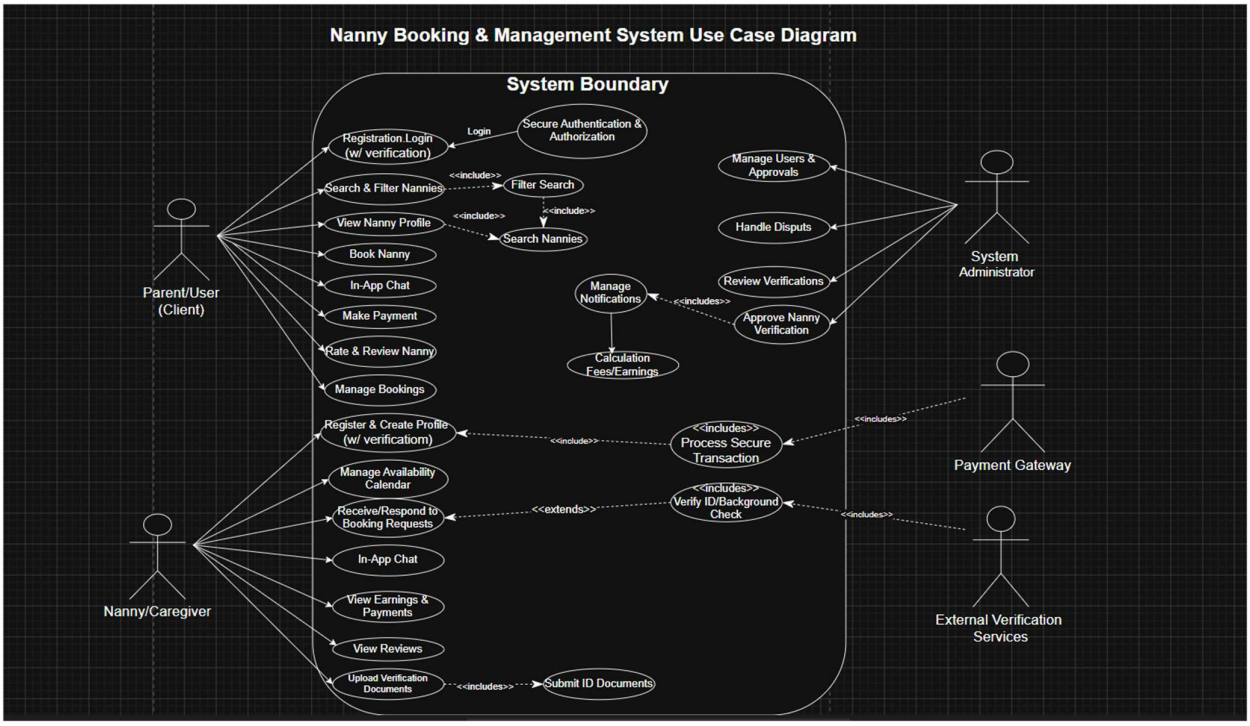
- Evaluate and rank nannies.

Nanny Functions:

- Make a professional profile and provide supporting documentation.
- Control the calendar of availability.
- Accept or deny reservations
- Talk to the parents
- View your earnings and payment history.
- Get reviews and ratings

System Functions:

- Secure user authentication and role-based access
- Real-time notifications
- Review and rating system
- Booking and payment management



(draw.io, 2025)

3. FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

3.1 FUNCTIONAL REQUIREMENTS

a) Depending on their role (Parent or Nanny), users will be able to register and log in.
b) Parents will use a variety of parameters to seek and filter nannies.
c) Detailed, verifiable profiles must be created and maintained by nannies.
d) Booking requests with an acceptance/rejection procedure will be supported by the system.
e) Secure real-time in-app messaging will be offered by the system.
f) A third-party gateway (Paystack) must be used to securely process payments.
g) After a service is completed, users will be able to evaluate and rate nannies.
h) When significant events occur, the system will promptly notify users.

3.2 NON-FUNCTIONAL REQUIREMENTS

- Security: HTTPS, password hashing, data encryption, secure payment processing.
- Usability: Simple, responsive design for Android, iOS, and the web.
- Performance: Quick reaction times (less than three seconds for page loads and searches).
- Reliability: High availability and frequent backups.
- Scalability: The ability to accommodate a growing user base over time.
- Compatibility: Support for multiple platforms.

4. LOGICAL SYSTEM MODEL

Input/Process/Output Table:

GUI/User Interface	System Process (Method/Function)	Entity/ Table (from Database)	Input	Output
Login Screen	User Authentication	User	Email/Phone + Password	Success/Failure message + Dashboard access
Nanny Search Page	Filter Nannies	Booking	Search criteria (location, date, rate, rating)	List of matching nanny profiles
Booking Request Screen	Create booking request	Booking	ParentID, NannyID, DateTime, Duration	Booking confirmation + Notification
Chat Interface	Send/Receive message	ChatMessage	Message text, SenderID, ReceiverID	Message saved + delivered to recipient
Payment Screen	Process payment	Payment_Booking	BookingID, Amount, Payment details	Payment receipt + status

Nanny Dashboard	Manage availability	Nanny_Profile	Availability calendar updates	Updated calendar

High-Level Data Flow:

- The system begins when a parent logs into the platform and searches for available nannies using filters such as location, availability, and hourly rate. The system retrieves matching nanny profiles from the database and displays them to the user. The parent then sends a booking request to the selected nanny. The nanny can accept or reject the request. Once accepted, both users can communicate through the in-app chat system. The parent proceeds with secure payment processing through the payment gateway. After service completion, the parent may leave a review and rating for the nanny, which updates the nanny's overall profile rating.

- Parents → Search → View Profiles → Send Booking Request → Nanny responds → Chat → Payment → Service Delivery → Review → Rating update.

5. ENTITY RELATIONSHIP TABLE

Entity	Primary Key	Key Attributes	Relationships
User	userID	fullName, email, phone, passwordHash, role ("parent"/"nanny")	1:1 with Nanny (for nannies)
NannyProfile	nannyProfileID	userID (FK), bio, experienceYears, hourlyRate, availabilityCalendar, verificationStatus, photoURL, averageRating	1:M with Booking
Booking	bookingID	parentID (FK), nannyID (FK), dateTime, duration, status, location	1:1 with Payment, 1:0..1 with Review
Payment	paymentID	bookingID (FK), amount, transactionID, status, timestamp	1:1 with Booking
ChatMessage	messageID	senderID (FK), receiverID (FK), content, timestamp, isRead	M:M between Users
Review	reviewID	bookingID (FK), reviewerID (FK), rating (1-5), comment, createdAt	1:1 with Booking

6. CLASS DIAGRAMS

Class Diagram Table:

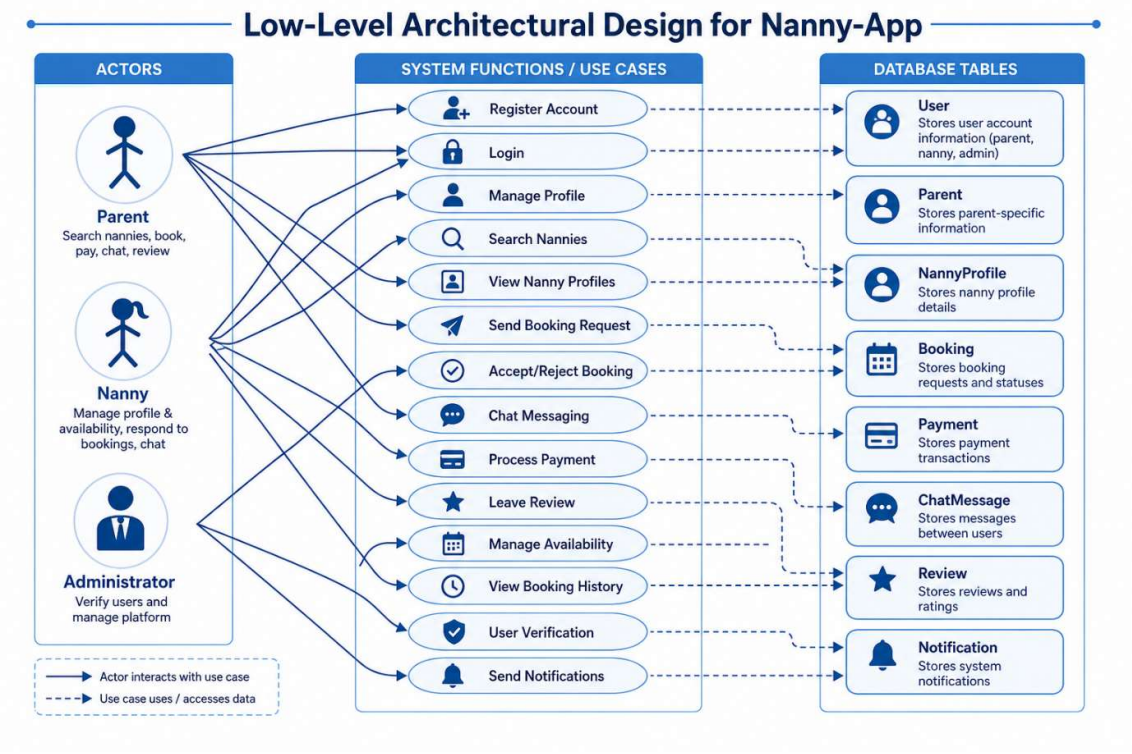
Name of entity (UML Class)	Properties of entity (UML Class)	Related to
User (abstract)	userID: int fullName: string (50) email: string (50) phone: string (15) passwordHash: string (255) role: string (10) - "parent" or "nanny"	Parent, Nanny (generalisation)
Parent	(inherits all User attributes) emergencyContact: string (15) numberOfChildren: int	Booking (1 to many)
Nanny	(inherits all User attributes) bio: string (500) experenceYears: int hourlyRate: decimal availabilityCalendar: string (JSON/text) verificationStatus: string (20) photoURL: string (200) averageRating: decimal	Nanny Profile (1 to 1, merged here), Booking (1 to many)

Booking	bookingID: int parentID: in (FK) nannyID: int (FK) dateTime: datetime duration: decimal (hours) status: string (e.g., pending, confirmed, cancelled) location: string (200)	Parent, Nanny, Payment (1 to o..1)
Payment	paymentID: int bookingID: int (FK) amount: decimal paymentMethod: string (30) transactionID: string (100) PaymentStatus: string (20) Timestamp: datetime	Booking
ChatMessage	messageID: int senderID: int (FK to User) recieverID: int (FK to User) content: string (1000) timestamp: datetime isRead: bool	User (sender, receiver)
Review	reviewID: int bookingID: int (FK) reviewerID: int (FK to User) rating: int (1-5) comment: string (1000) createdAt: datetime	Booking, User (reviewer)

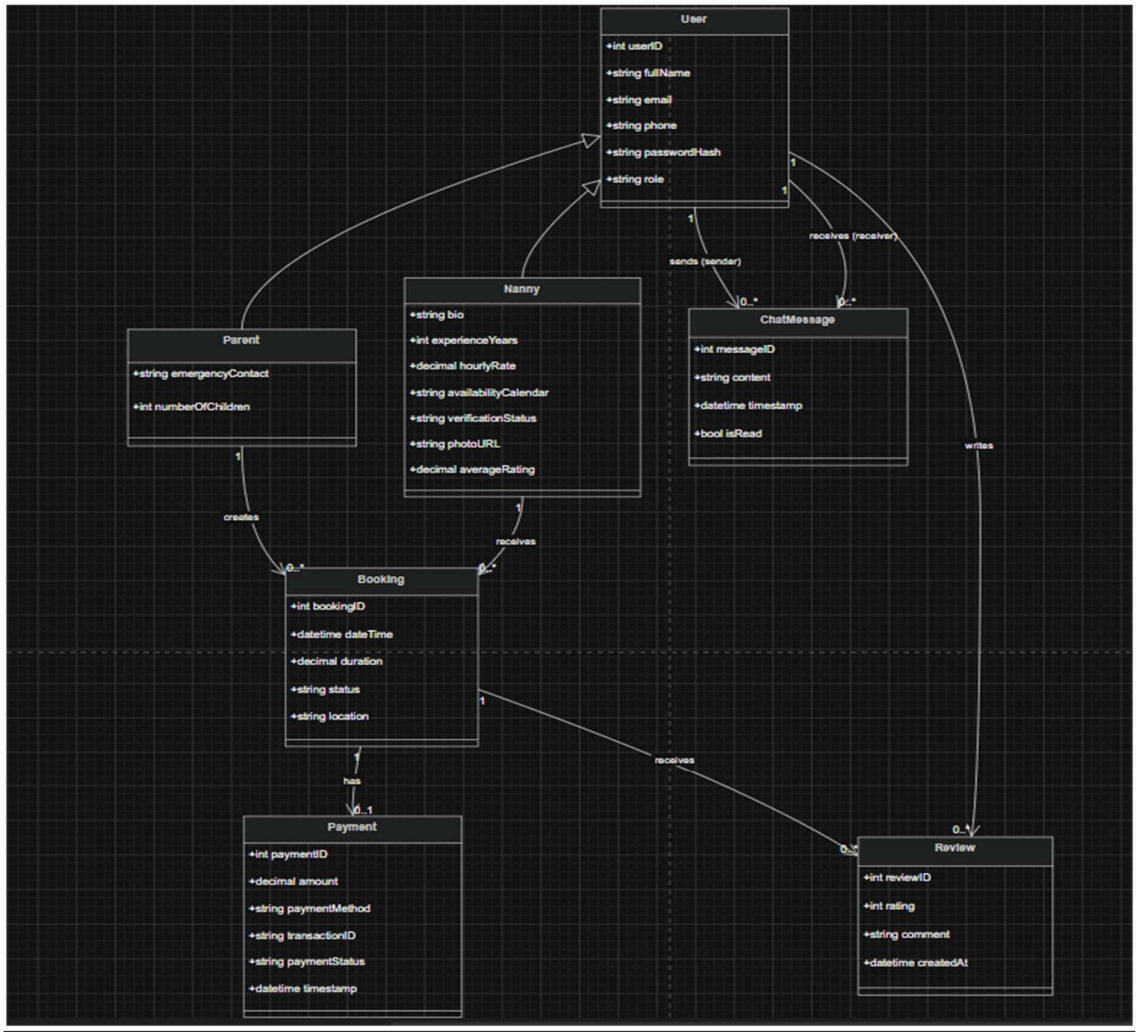
Interpretation:

- User is an abstract class that extends Parent and Nanny.
- Each parent can make many bookings, and each nanny can have multiple bookings.
- A booking can have one payment (optional 1-to-0..1).
- ChatMessage connects two user instances (sender and recipient).
- The review is optional but included to meet the rating/review criteria; it refers to a Booking and the User who wrote it.

Low-Level Architectural



Domain Class Diagram



(draw.io, 2025)

7. REFERENCES

- draw.io, 2026. *Flowchart Maker and Online Diagram Software*. [online] Available at: https://app.diagrams.net/ [Accessed 13 May 2026].
- Figma, 2026. *Figma Collaborative Interface Design Tool*. [online] Available at: https://www.figma.com/ [Accessed 13 May 2026].
- Paystack, 2026. *Online Payment Processing Platform*. [online] Available at: https://paystack.com/ [Accessed 13 May 2026].
- MySQL, 2026. *MySQL Database Management System*. [online] Available at: https://www.mysql.com/ [Accessed 13 May 2026].
- Node.js, 2026. *Node.js Documentation*. [online] Available at: https://nodejs.org/ [Accessed 13 May 2026].
- Visual Studio Code, 2026. *Visual Studio Code Documentation*. [online] Available at: https://code.visualstudio.com/ [Accessed 13 May 2026].
- Sommerville, I., 2016. *Software Engineering*. 10th ed. Harlow: Pearson Education.
- Pressman, R.S. and Maxim, B.R., 2020. *Software Engineering: A Practitioner's Approach*. 9th ed. New York: McGraw-Hill Education.
- The Independent Institute of Education, 2026. *Work Integrated Learning 3A – XISD5319 Module Manual*. Johannesburg: The Independent Institute of Education.
- Knights Team, 2026. *Project Plan – Nanny-App*. Unpublished academic project report.